Search and replace

Pattern: The pattern to search for. i.e. regular expression.
Match: The actual text that satisfies the pattern

Keys

```
%     find matching brace

*      Find next match for word under cursor. ie. \<{cword}\>
#      Find previous match for word under cursor.
g*     Same as *, but without the <>
g#     Same as #, but without the <>

[I      Show all matches for word under cursor (capital "i")
n      Next match
N      Previous match

gd     Go to declaration. May be local or global.
gD     Go to global declaration.
```

Commands

```
// Help
:help pattern             Regular expressions
:help search-commands     / and ? command
:help substitute          :substitute command


" Search: Search
"
:/<regex>           search forward
/<regex>            same
:?<regex>           search backward
?<regex>            same

:global/<regex>          show result in list
:g/<regex>/              shorthand
:g/<regex>/{cmd}         execute command
:g/<regex>/p             default (print)
:g/<regex>/normal dw     e.g. delete matched word



" Substitute: substitute matches
"
:s/<regex>/new text/                substitutes the first match in the
selected line with the new text

" Ranges
:11,23s/<regex>/new text/        from line 11 upto line 23, all inclusive
:.,$s/<regex>/new text/             from current position to end of file
:%s/<regex>/new text/              from entire file.
:'<,'>s/<regex>/new text/          from visual selection

" Flags
:s/<regex>/new text/gi              Multiple flags
:s/<regex>/new text/g               [g]lobal. Substitutes all matches of the
line with the new text
:s/<regex>/new text/c               [c]onfirm with a prompt
:s/<regex>/new text/i               [i]gnore case
:s/<regex>/new text/e               Silent [e]rror. By default, substitute
reports an error if there is no match.
```

==========

Search command

Search for pattern

:help search

Function

```
"" search({pattern}[, {flags}[, {stopline}[, {timeout}]]])
" Search for pattern from the given position
" - moves cursor to first match. Use 'n' flag to disable
" - returns line number
search("Hello")           "
let a=search("Hello")     " a=line number of match
                          " 0 if not found

" Pattern: regular expression
search("^Hello")
search("Hello\sThere")     " No need to escape special character

" Flags
search("Hello", 'b')
search("Hello", "bc")    " multiple
```

Flags

```
b        search backwards
c        accept match under cursor
e
n        do not move cursor
```

==========

Patterns

Regex

```
:help pattern-overview      Help of special characters

:help \^               Help for ^
:help \zs              Help for \zs
:help \ze
:help \@=


^      Beginning of line
$      End of line
\<     Beginning of word. Next character should be a word character [a-zA-
Z0-9_]
\>     End of word. Preceding character should be a word characater


.      any character
[a-z]
[^a-z]      NOT
\a          [a-zA-Z]
\A          non alphabet (in general, upper case is 'non')
\d          [0-9]
\s          white space
\l          [a-z]. Lowercase
\u          [A-Z]. Uppercase
\w          [a-zA-Z0-9_]. Word character
[\w\s]      Does not work. Cannot use special characters in [].
```

```
*               zero or more of the preceding character. i.e. \{0,}
\+              one or more. i.e. \{1,}
\=              zero or more. Unlike *, doesn't match empty lines and meta-
characters.
\?              zero or one. i.e. \{0,1}
\{n,m}          matches from n to m times, as many as possible.
\{n}            exactly n times.
\{,m}           at most m times, as many as possible.
\{n,}           at least n times, as many as possible.


\(\)            Grouping and capturing

                The grouped part can be referenced when replacing.
                \1 for first group, \2  for second group, etc.
                e.g. s/\(\d\+\)/Number:\1/
                   This converts "123" to "Number:123"
\1              First grouped item
\2
%\(\)           Group, but without capturing reference


abc\(def\)\@=          Positive look ahead. Matches lines containing abcdef,
but matching only abc.
abc\(def\)\@=gh        Never matches, since it has to be both abcdef and
abcgh.

abc\(def\)\@!          Positive lookahead. Matches any abc not followed by
def. e.g. Matches 'abc' of abcg, abcf
abc\(def\)\@!def       Never matches, since it looks both for abc not followed
by def (due to \@!) and followed by def (due to the last def).


\|               alternation
                  e.g. s/\(Hello\|Goodbye\)/Greeting:\1/

\c          Ignore case. Same as :set ignorecase
\C          Match case. Same as :set noignorecase


\zs     Set start of match. Match begins here.
\ze     Set end of match. Match ends here.
        e.g s/He\zsllo\zeo/LL   changes Hello -> HeLLo
```

Magic

magic: determines which characters are interpreted with special meaning.
- \m = same as 'set magic'

```
" Keys
"
/a\|b           Use default. Default depends on the 'magic' option.
/\va|b          very magic
/\ma\|b         magic
/\Ma\|b         no magic
/\Va\|b         very nomagic

s/\va/          very magic in search

:set magic      Use \m by default
:set nomagic    Use \M by default
```

From

left=most magic. Prioritize special meanings, and literals need to be escaped.
right=least magic. Only backslash has special meaning.

	\v (very magic)	\m (magic)	\M (no magic)	\V (very nomagic)
$=EOF . = any character * = any number of previous ~ = ? <> = word boundary {n,m} = match n to m atoms	$. * ~ <> {}	$. * ~ \<\> \{}	\$ \. * \~ \<\> \{}	\$ \. * \~ \<\> \{}
() = group | = alternate	() |	\(\) |	\(\) |	\(\) |
Special characters \a = alphabetic \w	\a \w	\a \w	\a \w	\a \w
Literals . = literal dot \ = literarl backslash	\. \\	\. \\	. \\	. \\

==========

Tricks

Search word under cursor

```
/<C-R><C-A>                word under cursor
/<C-R><C-W>                CWORD under cursor
```

Ignore case

```
/\chello              # Search
s/\chello/HELLO/      # Replace
```

Search for literal

Use very not-magic

```
/\V(Hello.)
```

Search repeated character or word

```
- Character
/a\*          0 or more a
/\va*         Magic
/a\+          1 or more a
/\va+         Magic
/a\{2,4}      2,3, or 4 a
/\va{2,4}     Magic


- Word
/\(abc\)\*     0 or more abc
/\v(abc)*
/\(abc\)\+     1 or more abc
/\v(abc)+
/\(abc\)\{2,4}  2,3, or 4 abc
/\v(abc){2,4}   Magic
```

Peak character or word

```
- Character
/a\?          0 or 1 a
/\va?         Magic

- Word
/\(abc\)\?
/\v(abc)?
```

Trim whitespace

```
%s/ \+/ /g
```

Insert new line

```
%s/Hello/\r/g            Replace Hello with a new line.
%s/\t/ /g
%s/\n/ /g
```

Match exact word

```
/\<function\>/
/\v<function>/    Very magic
```

Add spaces around a word

```
" Regexp for adding spaces
%s/\(function([a-zA-Z0-9\, _]*)\){/\1 {/igc          Turn "func
tion(){" into "function() {" i.e. add a space.
%s/\v(function([a-zA-Z0-9\, _]*)){/\1 {/igc          Very magic

%s/^(\s*)if(/\1if (/\g                              Turn "if(" into "if ("
%s/\v^(\s*)if(/\1if (/\g                              Turn "if(" into "if ("
```

Comment out selection

```
'<,'>s/^/#/                    Comment (#) out selection
'<,'>s/^#//                    Remove comment
```

Enclose in quotes

```
%s/\([a-zA-Z0-0_][a-zA-Z0-9_ ]\+\):/"\1":/gc         Enclose in double quotes
```